

Praktikum 8: Kürzeste Wege

Aufgabe 1: Dijkstra – Reindeer Maze

Das Rätsel „Advent of Code 2024, Tag 16“ (<https://adventofcode.com/2024/day/16>) beschreibt einen Rentierparcours: Ein Rentier startet an Position S mit Blickrichtung Ost. Es kann sich in einem Schritt vorwärts bewegen (Kosten: 1) oder um 90° drehen (Kosten: 1000). Gesucht ist der günstigste Weg von S zum Zielfeld E.

Das Beispiel-Labyrinth:

```
#####
# .....#...E#
#.#.###.#.###.#
#.....#.#...#.#
#.#.###.#####.#.#
#.#.#.....#.#
#.#.#####.###.#
#.....#.#
###.#.#####.#.#
#...#.....#.#.#
#.#.#.###.#.#.#
#.....#...#.#.#
#.#.###.#.#.#.#
#S..#.....#...#
#####
```

- Modellierung:** Ein Zustand ist ein Tripel (Zeile, Spalte, Richtung) mit Richtung $\in \{N, O, S, W\}$. Eine Kante existiert beim Vorwärtsbewegen (Kosten 1) und beim Drehen (Kosten 1000). Wie viele Knoten und Kanten hat der vollständige Graph für eine Karte mit R Zeilen und C Spalten (ohne Wände)? Welche Darstellung (Adjazenzliste oder -matrix) ist geeigneter?
- Dijkstra:** Bauen Sie den Graphen auf und nutzen Sie `dijkstra()` aus `vorlesung/L08_graphen/graph.py`, um den günstigsten Weg von S (Startrichtung Ost) nach E zu finden. Geben Sie die Gesamtkosten und den Pfad aus.
- AoC:** Melden Sie sich bei „Advent of Code“ an und laden Sie Ihren eigenen Parcours herunter. Ermitteln Sie die minimalen Kosten.

Aufgabe 2: Bellman-Ford

Der Bellman-Ford-Algorithmus findet kürzeste Wege auch bei negativen Kantengewichten und erkennt negative Zyklen. Der folgende Graph entspricht dem Vorlesungsbeispiel:

Von	Nach	Gewicht
a	b	9
a	d	4
b	c	1
b	d	2
c	b	5
c	e	4
d	c	-2
d	e	2

- Implementierung:** Implementieren Sie eine Funktion `bellman_ford(graph, start_name)` analog zu `dijkstra()`, die einen beliebigen Graphen (Unterklasse von `Graph`) als Parameter entgegennimmt. Nutzen Sie `graph.all_vertices()` und `graph.all_edges()` für den Zugriff auf den Graphen. Die Funktion gibt ein Tupel (`distance_map`, `predecessor_map`, `has_negative_cycle`) zurück. Orientieren Sie sich am Algorithmus aus der Vorlesung: $|V| - 1$ Durchläufe über alle Kanten zur Relaxation, danach ein abschließender Prüfdurchlauf zur Erkennung negativer Zyklen.
- Test:** Bauen Sie den Graphen aus der Tabelle auf und führen Sie Bellman-Ford von Knoten **a** aus. Geben Sie für jeden Knoten die kürzeste Distanz und den vollständigen Pfad aus.
- Negativer Zyklus:** Fügen Sie die Kante `e → b` mit Gewicht `-8` hinzu. Was gibt `has_negative_cycle` zurück? Erläutern Sie, welche Prüfung im Algorithmus den Zyklus erkennt, und zeigen Sie den betroffenen Zyklus im Graphen.
- Komplexität:** Bellman-Ford hat die Zeitkomplexität $O(|V| \cdot |E|)$, Dijkstra mit Min-Heap $O(|V| \log |V| + |E|)$. Wann ist Dijkstra vorzuziehen, und wann muss Bellman-Ford eingesetzt werden?

Musterlösung Praktikum 8: Kürzeste Wege

1 Dijkstra – Reindeer Maze

Die vollständige Lösung befindet sich im Repository unter `praktika/08_kuerzeste_wege/aufgabe1_dijkstra.py`.

a) Modellierung

Ein *Zustand* ist ein Tripel (r, c, d) mit Zeile r , Spalte c und Richtung $d \in \{N, O, S, W\}$ (kodiert als 0, 1, 2, 3). Die Kanten eines Zustands:

- **Vorwärtsbewegen:** $(r, c, d) \rightarrow (r', c', d)$ mit Gewicht 1, falls das Zielfeld kein Hindernis ist.
- **Drehen links/rechts:** $(r, c, d) \rightarrow (r, c, d')$ mit Gewicht 1000, wobei $d' = (d \pm 1) \bmod 4$.

Für eine Karte mit R Zeilen und C Spalten gelten folgende Schranken:

$$|V| \leq 4 \cdot R \cdot C$$

Jeder Zustand hat genau 2 Drehkanten und höchstens 1 Vorwärtskante, also $|E| \leq 3|V|$. Damit ist der Graph *dünn* (*sparse*); eine Adjazenzliste ist der Adjazenzmatrix deutlich überlegen: Die Matrix hätte $|V|^2$ Einträge, von denen nur $O(|V|)$ belegt wären.

Für das 15×15-Beispiellabyrinth ergibt sich:

$$|V| = 416, \quad |E| = 1052$$

b) Dijkstra

```
g, start, end = build_graph(GRID)

start_state = f"{start[0]},{start[1]},1" # Startrichtung Ost (d=1)
dist_map, pred_map = g.dijkstra(start_state)

# Alle 4 Richtungen am Zielfeld prüfen
end_vertices = [g.get_vertex(f"{end[0]},{end[1]},{d}") for d in range(4)]
best_end = min(end_vertices, key=lambda v: dist_map[v])
print(f"Gesamtkosten: {dist_map[best_end]}")

pfad = g.path(best_end.value, pred_map)
print(f"Pfad: {len(pfad)} Zustände ({len(pfad)-1} Schritte)")
```

Ausgabe für das 15×15-Beispiellabyrinth:

```
Graph: |V| = 416, |E| = 1052
Gesamtkosten: 7036
Pfad: 44 Zustände (43 Schritte)
Erste Schritte: 13,1,1 -> 13,1,0 -> 12,1,0 -> 11,1,0 -> 11,1,1 ...
Letzte Schritte: ... 3,13,0 -> 2,13,0 -> 1,13,0
```

Warum 4 Zielzustände? Das Zielfeld E ist unabhängig von der Ankunftsrichtung erreichbar. Es gibt also vier zulässige Endzustände (r_E, c_E, d) mit $d \in \{N, O, S, W\}$; der günstigste wird per `min()` ausgewählt.

c) AoC – Zeitmessung

```
import time
t0 = time.perf_counter()
g.dijkstra(start_state)
t1 = time.perf_counter()
print(f"Laufzeit (Beispiel): {(t1-t0)*1000:.3f}ms")
```

Für das 15×15 -Beispiel beträgt die Laufzeit unter 1 ms. Ein typisches AoC-Labyrinth hat $141 \times 141 = 19881$ Felder, also etwa $4 \times 19881 \approx 80\,000$ Knoten — rund 190-fach mehr. Da Dijkstra in $O(|V| \log |V| + |E|)$ wächst, ist eine Laufzeit im einstelligen Millisekundenbereich realistisch.

d) (Optional) Alle optimalen Felder – AoC Teil 2

Gesucht ist die Menge aller Felder (r, c) , die auf mindestens einem optimalen Pfad liegen. Vorgehen:

1. Dijkstra vom Startknoten $\rightarrow d_{\text{hin}}(v)$ für alle v .
2. Dijkstra auf dem *invertierten* Graphen vom Zielknoten $\rightarrow d_{\text{rück}}(v)$ für alle v .
3. Ein Zustand v liegt auf einem optimalen Pfad, wenn $d_{\text{hin}}(v) + d_{\text{rück}}(v) = d^*$ (Gesamtkosten).
4. Die Menge der Felder ergibt sich durch Projektion auf (r, c) .

Die Gesamtkomplexität bleibt $O(|V| \log |V| + |E|)$.

2 Bellman-Ford

Die vollständige Lösung befindet sich im Repository unter `praktika/08_kuerzeste_wege/aufgabe2_bellman_ford.py`.

Hinweis: Die Funktion `bellman_ford(graph, start_name)` ist als *eigenständige Funktion* implementiert und nutzt `graph.all_vertices()` sowie `graph.all_edges()`. Da beide Methoden in der abstrakten `Graph`-Klasse definiert sind, funktioniert sie mit jedem kompatiblen `Graph`-Objekt – eine Unterklasse ist nicht erforderlich.

a) + b) Implementierung und Test

Kernstruktur der Funktion:

```
def bellman_ford(graph, start_name):
    distance_map = {v: float('inf') for v in graph.all_vertices()}
    predecessor_map = {v: None for v in graph.all_vertices()}
    distance_map[graph.get_vertex(start_name)] = 0

    n = len(list(graph.all_vertices()))
    for _ in range(n - 1):
        for src_name, dst_name, weight in graph.all_edges():
            src = graph.get_vertex(src_name)
            dst = graph.get_vertex(dst_name)
            if distance_map[src] + weight < distance_map[dst]:
```

```

distance_map[dst] = distance_map[src] + weight
predecessor_map[dst] = src

# Prüfdurchlauf: jede weitere Verbesserung zeigt negativen Zyklus
has_negative_cycle = any(
    distance_map[graph.get_vertex(s)] + w < distance_map[graph.
        get_vertex(d)]
    for s, d, w in graph.all_edges()
)
return distance_map, predecessor_map, has_negative_cycle

```

Ausgabe für den (korrigierten) Beispielgraphen, Start a :

Negativer Zyklus: False

Kürzeste Distanzen:

```

a: d=0,   Pfad: a
b: d=7,   Pfad: a -> d -> c -> b
c: d=2,   Pfad: a -> d -> c
d: d=4,   Pfad: a -> d
e: d=6,   Pfad: a -> d -> e

```

Nachvollziehbarkeit (Vorlesungsbeispiel): Bellman-Ford benötigt zwei vollständige Durchläufe:

- 1. Durchlauf: Kanten von a relaxieren: $d[b] = 9$, $d[d] = 4$; dann $d \rightarrow c$ liefert $d[c] = 4 + (-2) = 2$; $d \rightarrow e$ liefert $d[e] = 4 + 2 = 6$.
- 2. Durchlauf: $c \rightarrow b$: $d[b] = 2 + 5 = 7$ (Verbesserung). Danach keine weiteren Änderungen.

Der 3. Durchlauf bringt keine Verbesserung mehr $\Rightarrow$ kein negativer Zyklus.

c) Negativer Zyklus

```

g.connect('e', 'b', -8)
dist2, pred2, neg_cycle2 = bellman_ford(g, 'a')
print(f"Negativer_Zyklus: {neg_cycle2}") # True

```

Ausgabe: Negativer Zyklus: True

Erklärung: Die neue Kante $e \rightarrow b$ schließt den Zyklus $b \rightarrow c \rightarrow e \rightarrow b$ mit Gewicht $1 + 4 + (-8) = -3 < 0$. Im abschließenden Prüfdurchlauf findet Bellman-Ford, dass sich $d[b]$ durch die Kante $e \rightarrow b$ noch weiter verringern lässt – ein Zeichen dafür, dass die Distanzen nicht stabil sind.

d) Komplexität

Im Beispielgraphen gilt $|V| = 5$ und $|E| = 8$ (bzw. $|E| = 9$ mit der negativen Zusatzkante).

Algorithmus	Zeitkomplexität	Voraussetzung
Dijkstra (Min-Heap)	$O(V \log V + E)$	keine negativen Gewichte
Bellman-Ford	$O(V \cdot E)$	negative Gewichte erlaubt

Wann Dijkstra, wann Bellman-Ford? Dijkstra ist für Graphen ohne negative Kantengewichte klar vorzuziehen: Für große, dichte Graphen ist $O(|V| \log |V| + |E|)$ deutlich effizienter als $O(|V| \cdot |E|)$. Bellman-Ford ist notwendig, wenn negative Kantengewichte auftreten (z. B. Wechselkursarbitrage, Routing-Protokolle mit negativen Metriken) oder wenn man prüfen will, ob überhaupt negative Zyklen existieren.